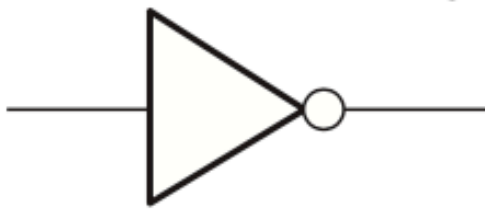


Boolean logic

Zain Merchant

Logic gate symbols

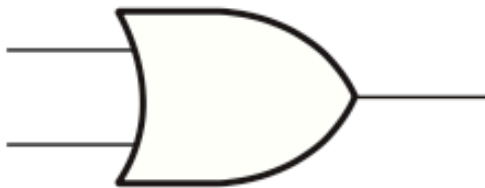
Six different logic gates will be considered in this chapter:



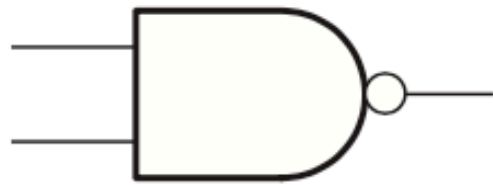
NOT gate



AND gate



OR gate



NAND gate



NOR gate



XOR gate

Truth tables

Truth tables are used to trace the output from a logic gate or logic circuit. The NOT gate is the only logic gate with one input; the other five gates have two inputs.

Although each logic gate can only have one or two inputs, the number of inputs to a logic circuit can be more than 2; for example, three inputs give a possible 2^3 (=8) binary combinations. And for four inputs, the number of possible binary combinations is 2^4 (=16). It is clear that the number of possible binary combinations is a multiple of the number 2 in every case. The possible inputs in a truth table can be summarised as shown below.

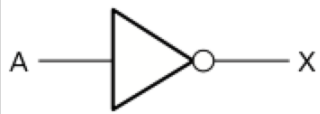
Inputs	
A	B
0	0
0	1
1	0
1	1

Inputs		
A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Inputs			
A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Functions of logic gates

NOT gate

Description	Truth table	How to write this	Symbol								
The output, X, is 1 if: the input, A, is 0	<table><tr><th>Input</th><th>Output</th></tr><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	Input	Output	A	X	0	1	1	0	X = NOT A (logic notation)	
Input	Output										
A	X										
0	1										
1	0										

AND gate

Description	Truth table	How to write this	Symbol																		
The output, X, is 1 if: both inputs, A and B, are 1	<table><tr><th colspan="2">Inputs</th><th>Outputs</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Inputs		Outputs	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1	X = A AND B (logic notation)	
Inputs		Outputs																			
A	B	X																			
0	0	0																			
0	1	0																			
1	0	0																			
1	1	1																			

OR gate

Description	Truth table	How to write this	Symbol																		
The output, X, is 1 if: either input, A or B, or both, are 1	<table><tr><th colspan="2">Inputs</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Inputs		Output	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1	X = A OR B (logic notation)	
Inputs		Output																			
A	B	X																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	1																			

NAND gate (NOT AND)

Description	Truth table	How to write this	Symbol																		
The output, X, is 1 if: input A AND input B are NOT both 1	<table><tr><th colspan="2">Inputs</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Inputs		Output	A	B	X	0	0	1	0	1	1	1	0	1	1	1	0	X = A NAND B (logic notation)	
Inputs		Output																			
A	B	X																			
0	0	1																			
0	1	1																			
1	0	1																			
1	1	0																			

NOR gate (NOT OR)

Description	Truth table	How to write this	Symbol																		
The output, X, is 1 if: neither input A nor input B is 1	<table><tr><th colspan="2">Inputs</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Inputs		Output	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0	X = A NOR B (logic notation)	
Inputs		Output																			
A	B	X																			
0	0	1																			
0	1	0																			
1	0	0																			
1	1	0																			

XOR gate

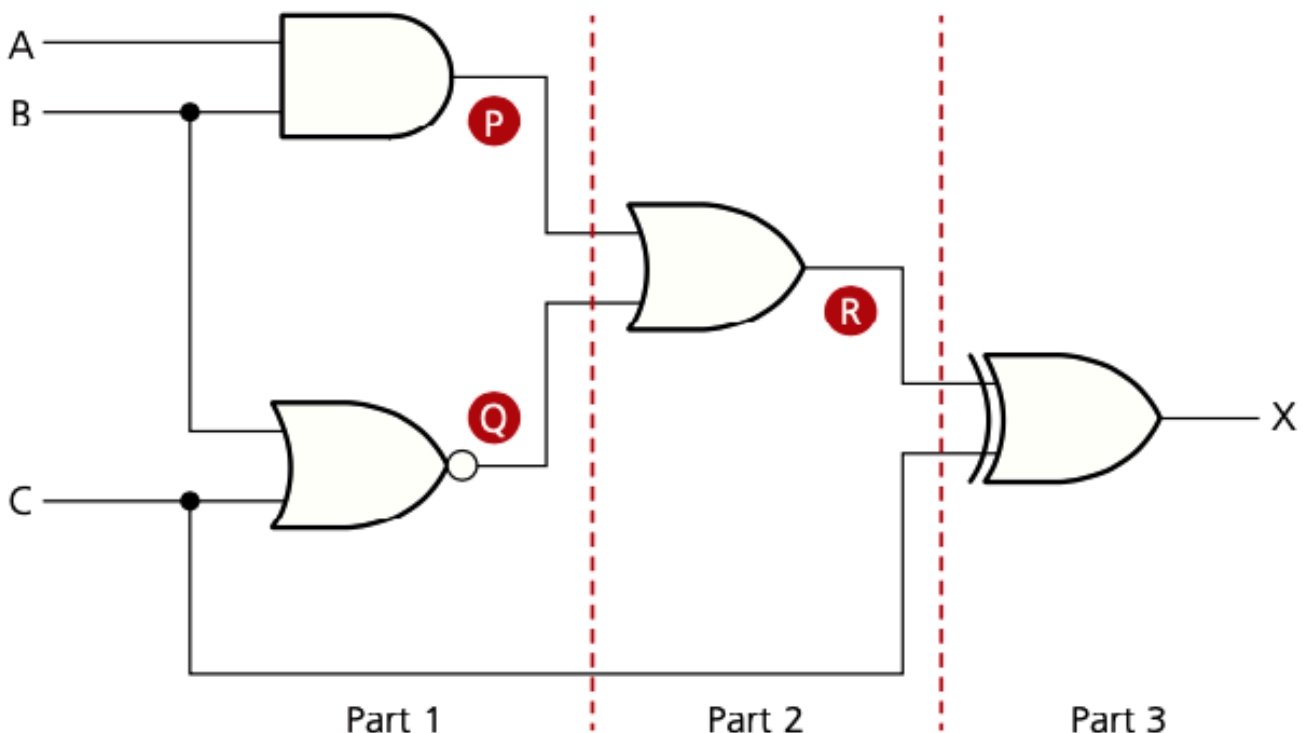
Description	Truth table	How to write this	Symbol																		
The output, X, is 1 if: (input A is 1 AND input B is 0) or (input A is 0 AND input B is 1)	<table><tr><th colspan="2">Inputs</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Inputs		Output	A	B	X	0	0	0	0	1	1	1	0	1	1	1	0	X = A XOR B (logic notation)	
Inputs		Output																			
A	B	X																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	0																			

Logic circuits, logic expressions, truth tables and problem statements

Example 1

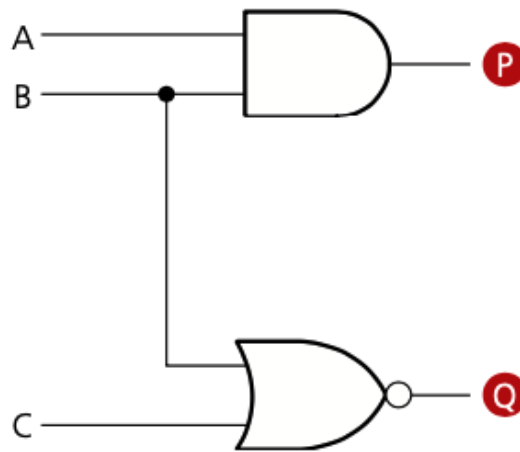
Zain Merchant

Produce a truth table for the following logic circuit (note the use of black circles at the junctions between wires):



There are three inputs to this logic circuit, therefore, there will be eight possible binary values that can be input.

To show stepwise how the truth table is produced, the logic circuit has been split up into three parts, as shown by the dotted lines, and intermediate values are shown as P, Q and R.



Part 1

This is the first part of the logic circuit; the first task is to find the intermediate values P and Q.

The value of P is found from the AND gate where the inputs are A and B. The value of Q is found from the NOR gate where the inputs are B and C. An intermediate truth table is produced using the logic function descriptions

input values			Output values	
A	B	C	P	Q
0	0	0	0	1
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	1
1	0	1	0	0
1	1	0	1	0
1	1	1	1	0

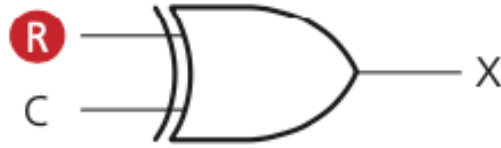


Part 2

The second part of the logic circuit has P and Q as inputs and the intermediate output, R:

This produces the following intermediate truth table. (Note: even though there are only two inputs to the logic gate, we have generated eight binary values in part 1 and these must all be used in this second truth table).

Inputs		Output
P	Q	R
0	1	1
0	0	0
0	0	0
0	0	0
0	1	1
0	0	0
1	0	1
1	0	1



Part 3

The final part of the logic circuit has R and C as inputs and the final output, X:

This gives the third intermediate truth table:

Inputs		Output
R	C	X
1	0	1
0	1	1
0	0	0
0	1	1
1	0	1
0	1	1
1	0	1
1	1	0

Putting all three intermediate truth tables together produces the final truth table, which represents the original logic circuit:

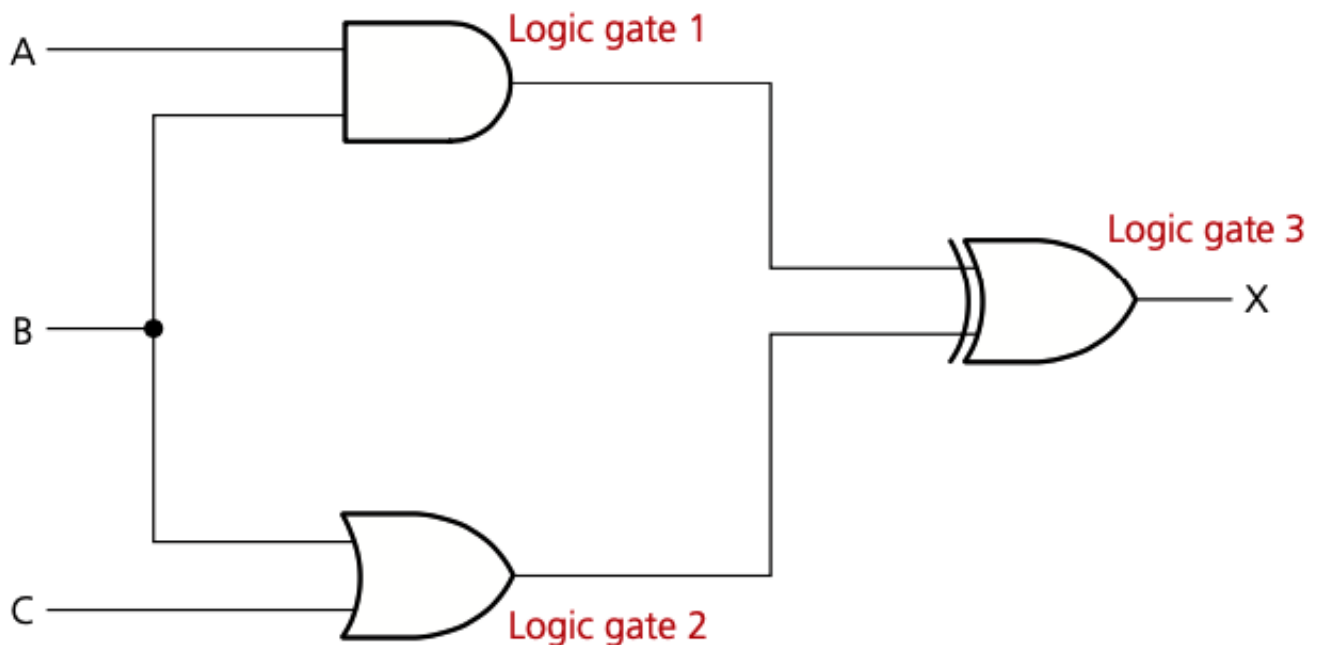
Input values			Intermediate values			Output
A	B	C	P	Q	R	X
0	0	0	0	1	1	1
0	0	1	0	0	0	1
0	1	0	0	0	0	0
0	1	1	0	0	0	1
1	0	0	0	1	1	1
1	0	1	0	0	0	1
1	1	0	1	0	1	1
1	1	1	1	0	1	0

The intermediate values can be left out of the final truth table, but it is good practice to leave them in until you become confident about producing the truth tables. The final truth table would then look like this:

Input values			Output
A	B	C	X
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Example 2

Write logic expressions from the following logic circuits:



The first action is to look at the gates connected to the inputs A, B and C:

logic gate 1: (A AND B)

logic gate 2: (B OR C)

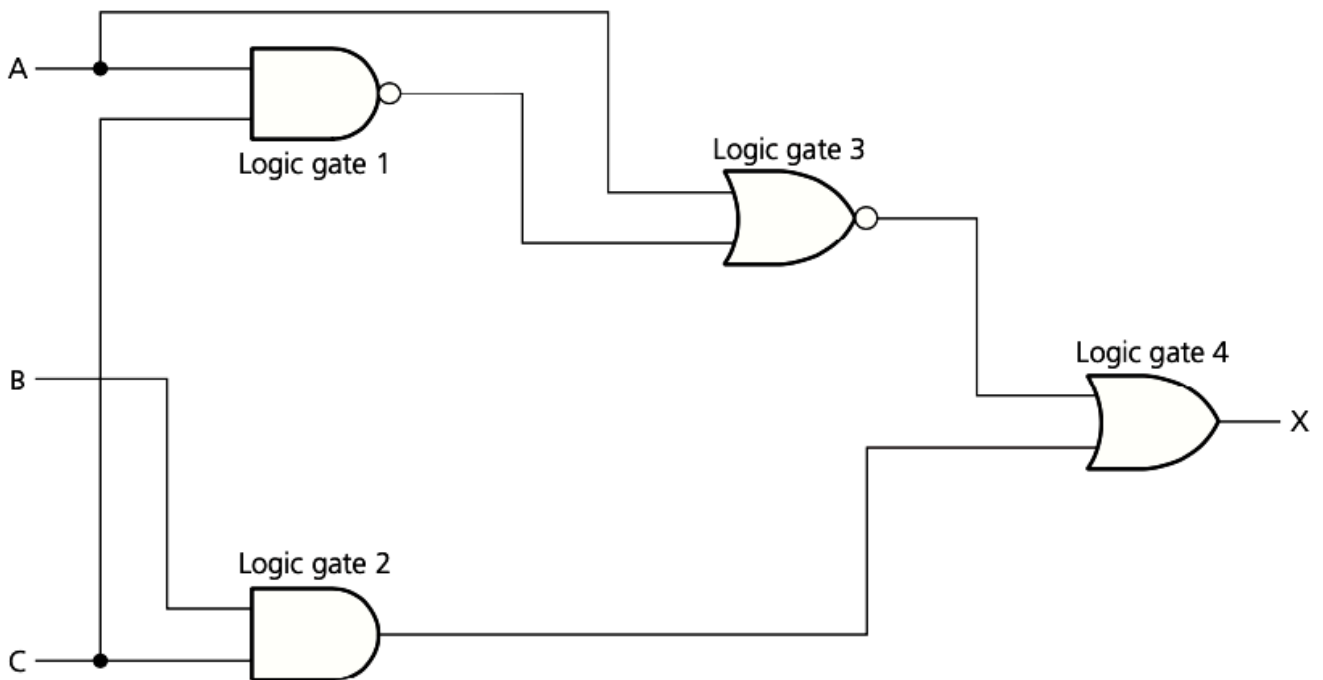
We then join these together using logic gate 3:

$[(A \text{ AND } B)] \text{ XOR } [(B \text{ OR } C)]$ which gives us the required logic expression.

(Note: the square brackets “[]” in the expression are not necessary and are used here just for clarity.)

This would be written as: $(A \text{ AND } B) \text{ XOR } (B \text{ OR } C)$

Example 3



Again, we will do this in the order of logic gates 1 and 2 first (connected to the three inputs):

logic gate 1: $(A \text{ NAND } C)$

logic gate 2: $(B \text{ AND } C)$

However, logic gate 3 is also connected to one of the inputs so that should be done next:

logic gate 3: $(\text{logic gate 1}) \text{ NOR } A$

If we replace (logic gate 1) by the logic expression above, we get:

$((A \text{ NAND } C) \text{ NOR } A)$ Finally, we can join all these together using:

logic gate 4: $((A \text{ NAND } C) \text{ NOR } A) \text{ OR } (B \text{ AND } C)$

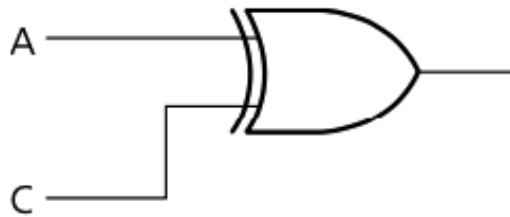
Example 4

A logic circuit can be represented by the following logic expression: $(A \text{ XOR } C) \text{ OR } (\text{NOT } C \text{ NAND } B)$

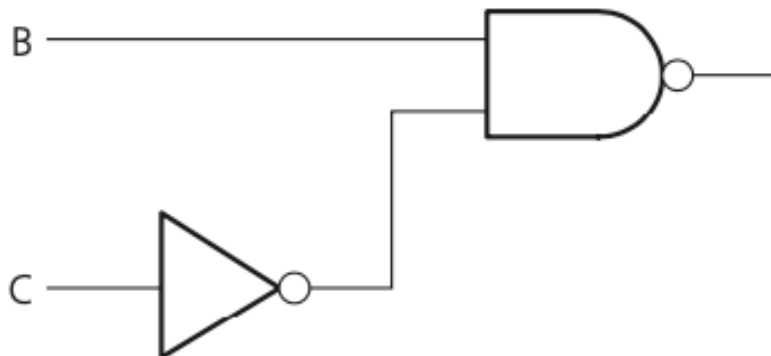
Produce a logic circuit and a truth table from the above statement.

In this example we have a connecting logic gate which is OR.

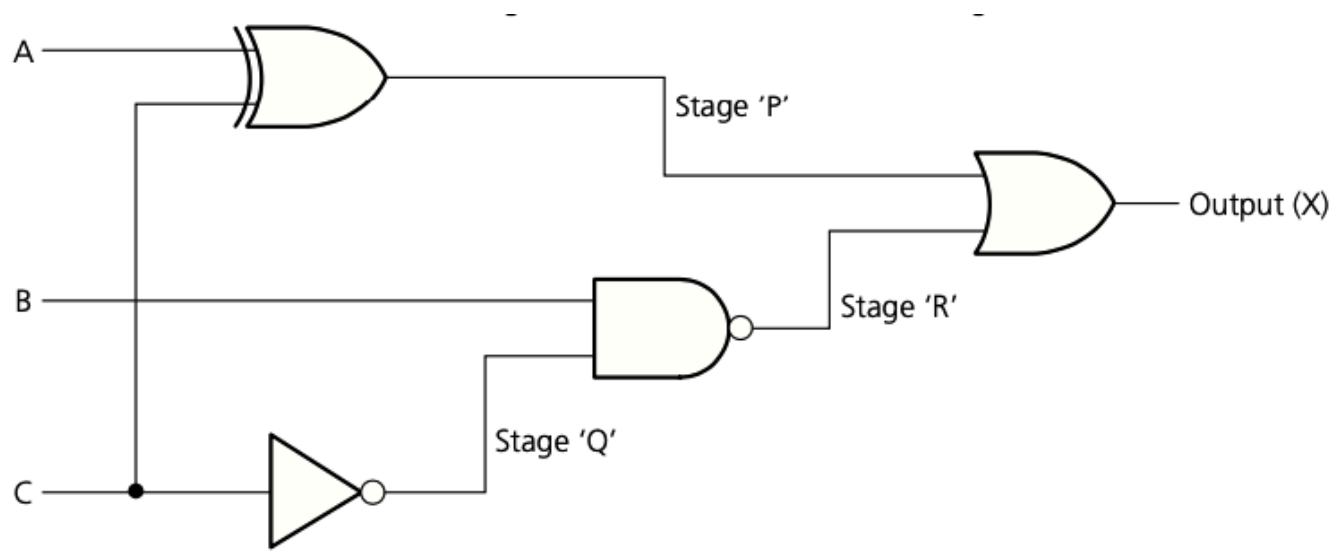
So, if we produce one half of the circuit from $(A \text{ XOR } C)$ we get:



The other half of the circuit is found from $(\text{NOT } C \text{ NAND } B)$:



If we now combine these together to form the final logic circuit:



The truth table is shown:

Input values			Values at stages:			Output
A	B	C	'P'	'Q'	'R'	X
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	0	1	0	0
0	1	1	1	0	1	1
1	0	0	1	1	1	1
1	0	1	0	0	1	1
1	1	0	1	1	0	1
1	1	1	0	0	1	1

Example 5

Look at the two truth tables below; in each case produce a logic expression and the corresponding logic circuit

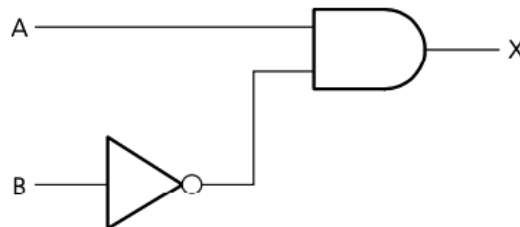
Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	1
1	1	0

To produce the logic statement, we only concern ourselves with the truth table row where the output value is 1. In this case, $A = 1$ and $B = 0$ which gives the logical expression:

$A \text{ AND NOT } B$

So we have the logic expression: $A \text{ AND NOT } B$

It is now possible to draw the corresponding logic circuit:



Example 6

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	0
1	1	1

This time we have **two** rows where the output is 1; this gives the following logical expressions:

(NOT A AND B)

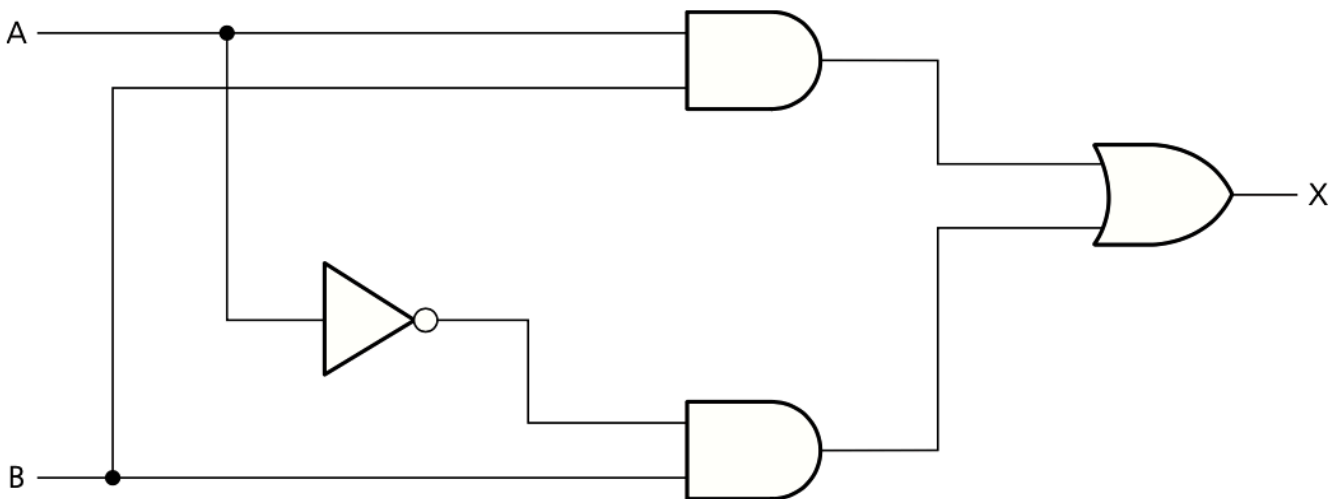
(A AND B)

We now join these together using an OR gate to give:

(NOT A AND B) OR (A AND B)

So we have the logic expression: (NOT A AND B) OR (A AND B)

It is now possible to draw the corresponding logic circuit:



Example 7

Which Boolean expression is represented by the following truth table?

Input values			Output
A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

We only need to consider those rows where the output is a 1. This gives us the following three logic expressions:

(NOT A AND NOT B AND NOT C)

(A AND NOT B AND NOT C)

(A AND B AND NOT C)

If we now join the three expressions with an OR gate, we end up with the final logic expression:

(NOT A AND NOT B AND NOT C) OR (A AND NOT B AND NOT C) OR (A AND B AND NOT C)

Example 8

Which logic expression is represented by the following truth table?

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

We only need to consider those rows where the output is a 1. This gives us the following four Boolean expressions:

(NOT A AND B AND C)

(A AND NOT B AND C)

(A AND B AND NOT C)

(A AND B AND C)

If we now join the four expressions with an OR gate we end up with the following logic expression:

(NOT A AND B AND C) OR (A AND NOT B AND C) OR (A AND B AND NOT C) OR (A AND B AND C)

Show that your logic expression in part 1 is the same as: $(B \text{ AND } C) \text{ OR } (A \text{ AND } C) \text{ OR } (A \text{ AND } B)$

To show that $(B \text{ AND } C) \text{ OR } (A \text{ AND } C) \text{ OR } (A \text{ AND } B)$ produces the same output as that shown in part 1 we need to produce a new truth table and show that the output is the same as the one in the given truth table:

A	B	C	B AND C	A AND C	A AND B	$(B \text{ AND } C) \text{ OR } (A \text{ AND } C) \text{ OR } (A \text{ AND } B)$	X
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	1	0	1	0	1	1
1	1	0	0	0	1	1	1
1	1	1	1	1	1	1	1

As the second truth table shows, the outputs from logic expressions are both the same; thus the logic expression $(B \text{ AND } C) \text{ OR } (A \text{ AND } C) \text{ OR } (A \text{ AND } B)$ gives the same output as the logic expression in part 1.

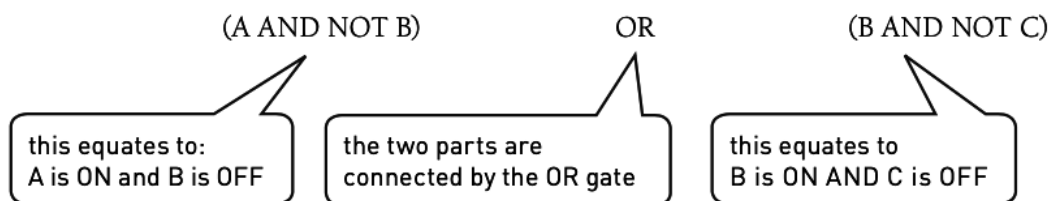
Example 9

A safety system uses three inputs to a logic circuit. An alarm, X, sounds if input A represents ON and input B represents OFF; or if input B represents ON and input C represents OFF.

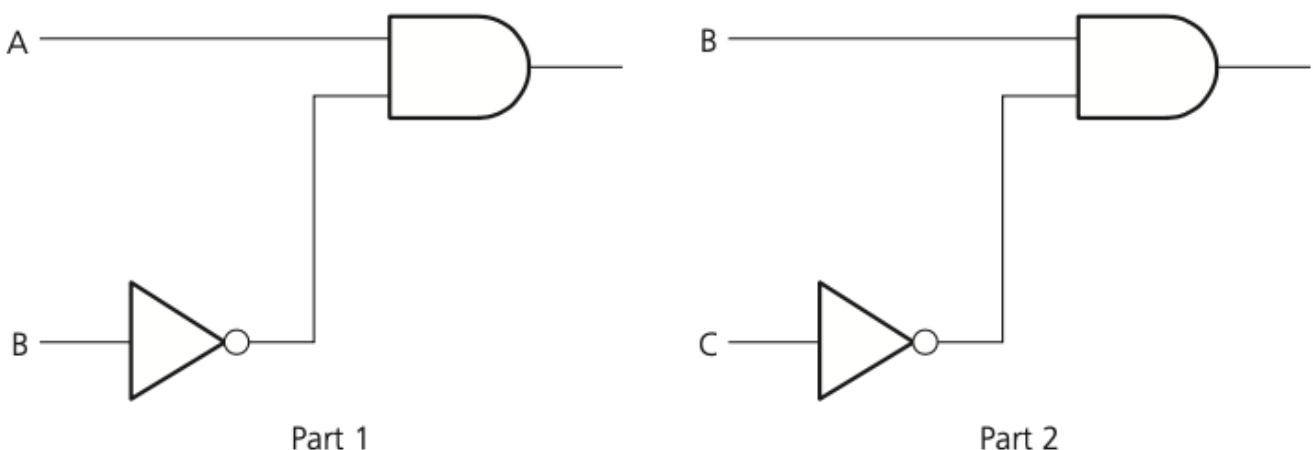
Produce a logic circuit and truth table to show the conditions that cause the output X to be 1.

The first thing to do is to write down the logic statement representing the scenario in this example. To do this, it is necessary to recall that ON = 1 and OFF = 0 and also that 0 is usually considered to be NOT 1.

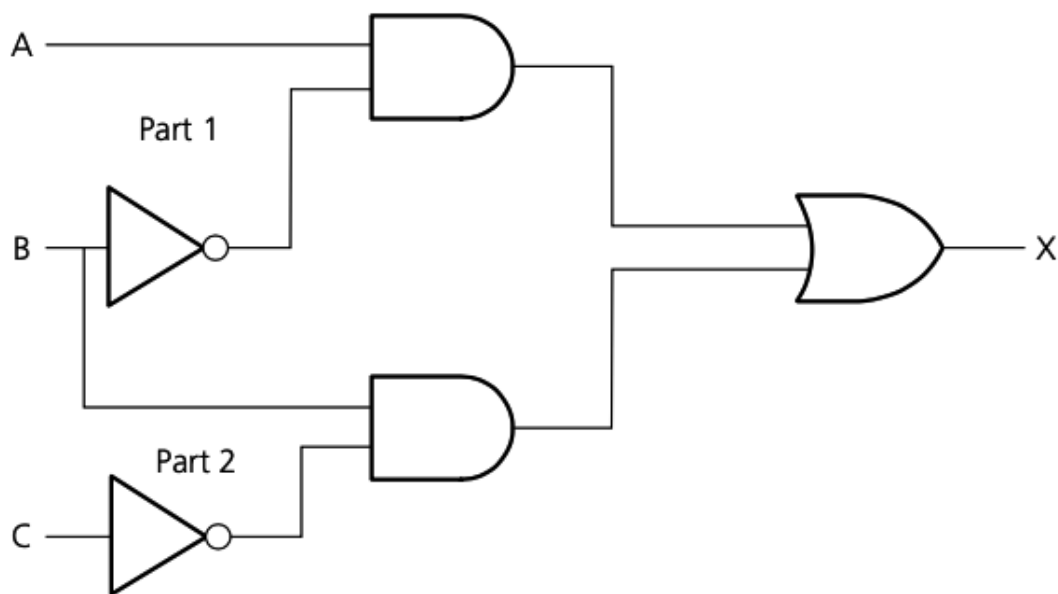
So we get the following logic expression:



The logic circuit is made up of two parts as shown in the logic expression. We will produce the logic gate for the first second part. Then join both parts together with the OR gate.



Now combining both parts with the OR gate gives us:



produce the truth table using the original logic expression; this second method has the advantage that it allows you to check that your logic circuit is correct.

Inputs			Intermediate values		Output
A	B	C	(A AND NOT B)	(B AND NOT C)	X
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	1	0	0	0

Example 10

A wind turbine has a safety system which uses three inputs to a logic circuit. A certain combination of conditions results in an output, X, from the logic circuit being equal to 1. When the value of $X = 1$ then the wind turbine is shut down.

The following table shows which parameters are being monitored and form the three inputs to the logic circuit.

parameter description	parameter	binary value	description of condition
turbine speed	S	0	turbine speed ≤ 1000 rpm
		1	turbine speed > 1000 rpm
bearing temperature	T	0	bearing temperature $\leq 80^{\circ}\text{C}$
		1	bearing temperature $> 80^{\circ}\text{C}$
wind velocity	W	0	wind velocity ≤ 120 kph
		1	wind velocity > 120 kph

The output, X, will have a value of 1 if any of the following combination of conditions occur:

Either turbine speed ≤ 1000 rpm and bearing temperature $> 80^{\circ}\text{C}$

Or turbine speed > 1000 rpm and wind velocity > 120 kph

Or bearing temperature $\leq 80^{\circ}\text{C}$ and wind velocity > 120 kph

Design the logic circuit and complete the truth table to produce a value of $X = 1$ when either of the three conditions above occur.

In this example, a real situation is given and it is necessary to convert the information into a logic expression and then produce the logic circuit and truth table. It is advisable in problems as complex as this to produce the logic circuit and truth table separately (based on the conditions given) and then check them against each other to see if there are any errors.

Stage 1

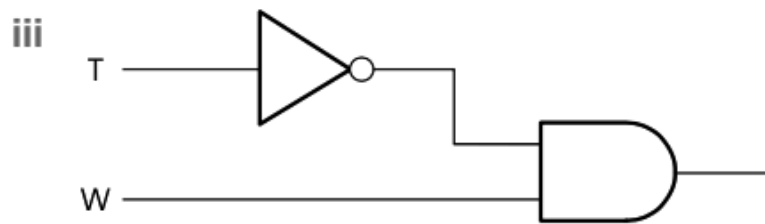
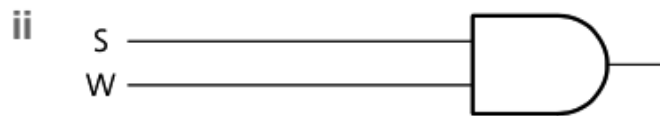
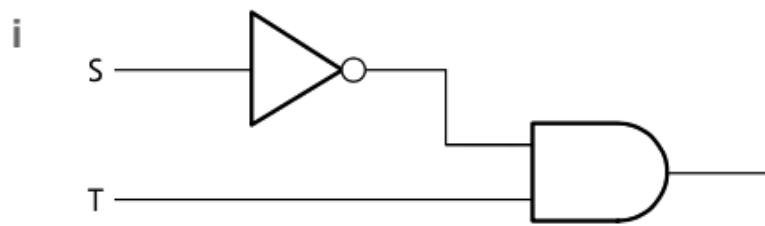
The first thing to do is to convert each of the three statements into logic expressions. Use the information given in the table and the three condition statements to find how the three parameters S, T and W are linked. We usually look for the key words AND, OR and NOT when converting actual statements into logic.

We end up with the following three logic expressions:

- turbine speed ≤ 1000 rpm and bearing temperature $> 80^{\circ}\text{C}$
logic expression: (NOT S AND T)
- turbine speed > 1000 rpm and wind velocity > 120 kph
logic expression: (S AND W)
- bearing temperature $\leq 80^{\circ}\text{C}$ and wind velocity > 120 kph
logic expression: (NOT T AND W)

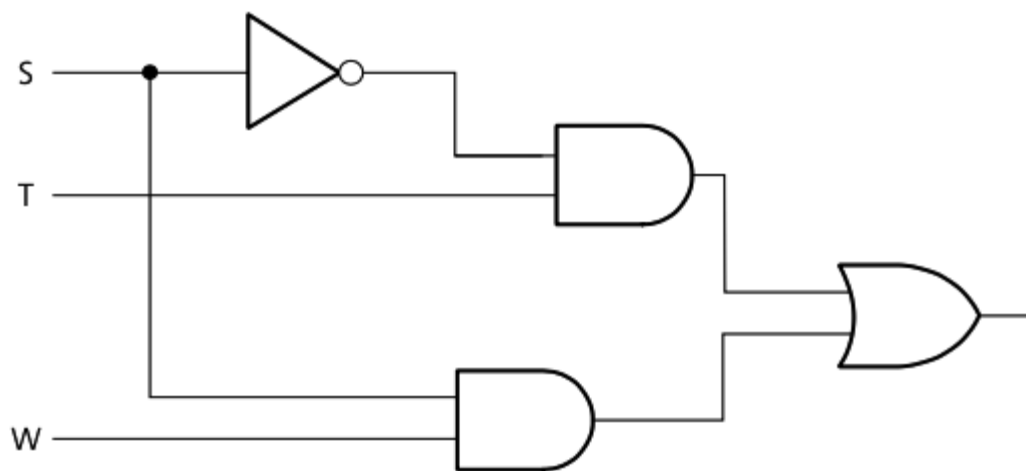
Stage 2

This now produces three intermediate logic circuits:

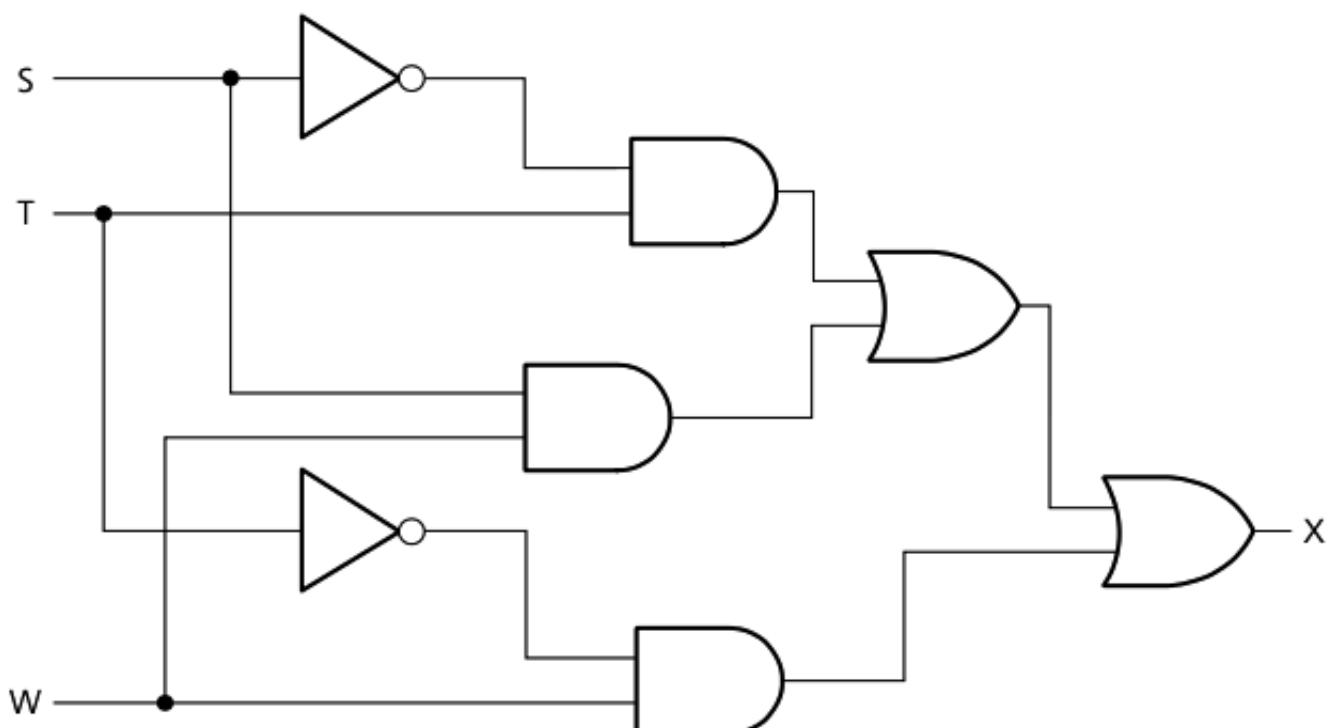


Each of the three original statements were joined together by the word 'OR'. Thus, we need to join all of the three intermediate logic circuits by two OR gates to get the final logic circuit.

We will start by joining i and ii together using an OR gate:



Finally, we connect the logic circuit in previous figures to obtain the answer:



The final part is to produce the truth table. We will do this using the original logic statement. This method has the bonus of allowing an extra check to be made on the logic circuit in Figure 10.32 to see whether or not it is correct. It is possible, however, to produce the truth table straight from the logic circuit in Figure 10.32.

There were three parts to the problem, so the truth table will first evaluate each part. Then, by applying OR gates, as shown below, the final value, X, is obtained:

- (NOT S AND T)
- (S AND W)
- (NOT T AND W)

We find the outputs from parts (i) and (ii) and then OR these two outputs together to obtain a new intermediate, which we will now label part (iv).

We then OR parts (iii) and (iv) together to get the value of X.

Inputs			Intermediate values				Output
S	T	W	(i) (NOT S AND T)	(ii) (S AND W)	(iii) (NOT T AND W)	(iv)	X
0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	1
0	1	0	1	0	0	1	1
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	1	0	1	1	1	1
1	1	0	0	0	0	0	0
1	1	1	0	1	0	1	1

Example 11

Consider the logic statement:

$$((A \text{ NOR } B) \text{ AND } C) \text{ NAND } (A \text{ OR NOT } B)$$

- Draw a logic circuit to represent the given logic statement.
- Complete the truth table for the given logic statement.

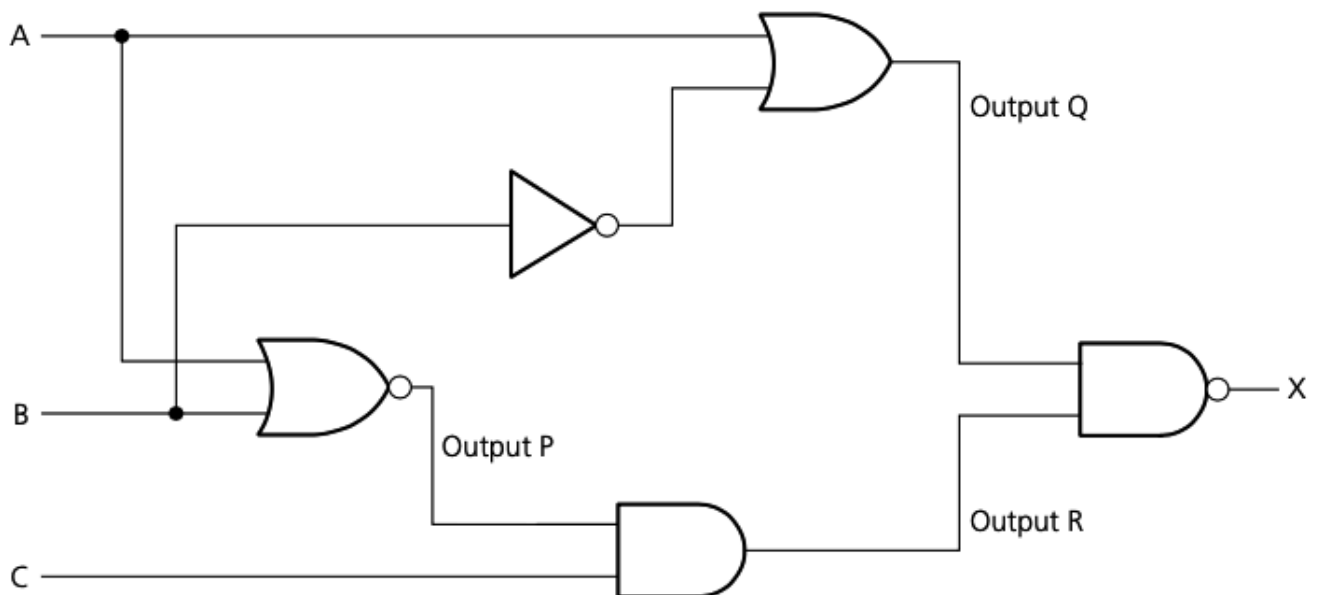
Input values			Working space	Output X
A	B	C		
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

First, we need to break down the logic statement. Assign:

1. $P = (A \text{ NOR } B)$
2. $Q = (A \text{ OR NOT } B)$
3. $R = (P \text{ AND } C)$

And then note that $X = R \text{ NAND } Q$

Now draw the logic gates for statements 1 to 3 and connect them together.



We can then fill out the truth table in stages, starting with P, Q and then R, followed finally by X:

Input Values			Working Space			Output X
A	B	C	P = (A NOR B)	Q = (A OR NOT B)	R = (P AND C)	
0	0	0	1	1	0	1
0	0	1	1	1	1	0
0	1	0	0	0	0	1
0	1	1	0	0	0	1
1	0	0	0	1	0	1
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1